IOStream

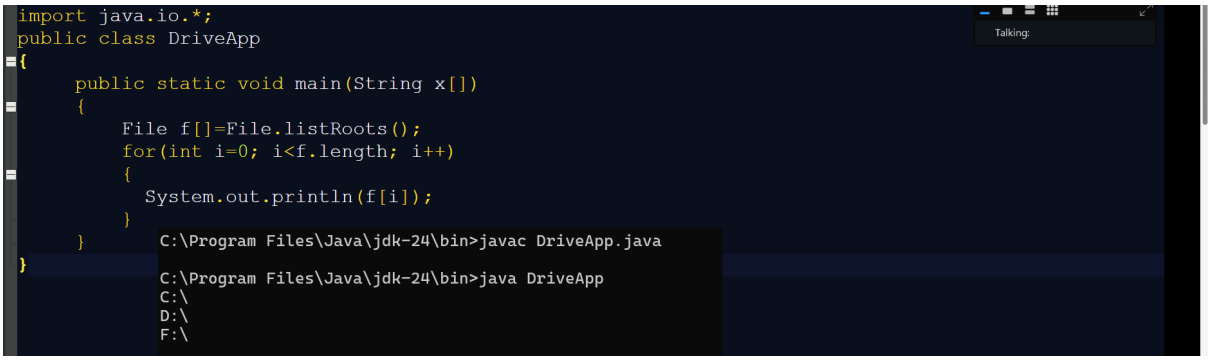
IOStream means input and output stream - here stream indicate to transfer data from one location to another location called as iostream
Here IOStream means transfer data from one memory to another memory called IOStream or send data from one machine to another machine called IOStreams.
If we want to work with iostream we have to known where send data or retrieve data
Means we need to know the location or path
If we want to access the path in java we have File class from java.io package

File class : File class is used for set location or access path, as well as create a folder using java , create a file using a java , access all drives or partitions from the machine as well as get a list of files and folders from the machine.

How to get list of drives from machine using File class

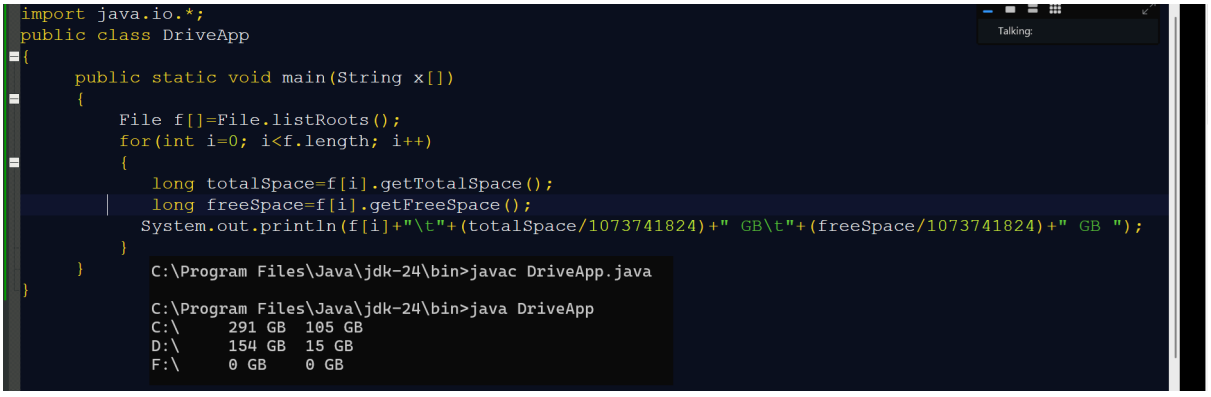
If we want to get list of drive from computer using a File we have method name as listRoots() and it is a static method of File class and return reference of File class array.

Syntax: File ref[]=File.listRoots()



If we want to get the total size of drive as well as free space of drive
We have getTotalSpace() method for fetch all space of driver as well as getFreeSpace() method for fetch free space of drive

long getTotalSpace(): this method is used for return total space of drive in bits format
long getFreeSpace(): this method is used for return space of drive in bits format



How to create folder using File class and How to check folder is present or not

---

If we want to create folder using a File class we have boolean mkdir() method and boolean exists()

boolean mkdir(): this method is used for create new folder and if folder created successfully return true otherwise return false

boolean exists(): this method is used for check file or folder exist on drive or not if exist then return true otherwise return false

```
import java.io.*;
public class CreateFolderApp
{
    public static void main(String x[])
    {
        File f= new File("D:\\janfebmarch123");

        if(f.exists())
        {  System.out.println("Folder is present");
        }
        else{
            boolean b=f.mkdir();
            if(b)
            { System.out.println("Foler created successfully...");
            }
            else
            { System.out.println("Some problem is there.............");
            }
        }
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac CreateFolderApp.java

C:\Program Files\Java\jdk-24\bin>java CreateFolderApp
Foler created successfully...

C:\Program Files\Java\jdk-24\bin>java CreateFolderApp
Folder is present

C:\Program Files\Java\jdk-24\bin>
```

How to get list of files and folder from a specified path

If we want to get list of files and folder from a specified path then we have inbuilt method name as listFiles() and this method return reference of File []

Syntax: File ref[]=fileref.listFiles()

boolean isDirectory(): this method is used for check path is directory or not if directory return true otherwise return false

boolean isFile(): this method checks if the path is file or not if file return true otherwise return false.

Example:

```
import java.io.*;
public class FetchFileFolderApp
{
    public static void main(String x[])
    {
        File f = new File("D:\\JUNE2026");
        File list[]=f.listFiles();
        for(int i=0; i<list.length; i++)
        {
            if(list[i].isFile())
            {
                System.out.println(list[i]);
            }
        }
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac FetchFileFolderApp.java

C:\Program Files\Java\jdk-24\bin>java FetchFileFolderApp
D:\JUNE2026\test
D:\JUNE2026\testgit
```

How to create file using a java program

If we want to create file using a java program we have method name as
boolean createNewFile(): this method is used for create new file and return true if file is created successfully otherwise return false

```
import java.io.*;
public class CreateFileApp
{
    public static void main(String x[])throws IOException
    {
       File f = new File("D:\\janfebmarch\\resume.doc");
       boolean b=f.createNewFile();
       if(b)
       { System.out.println("File created successfully....");
       }
       else
       { System.out.println("Some problem is there.............");
       }
    }
}
```

Output

```
C:\Program Files\Java\jdk-24\bin>javac CreateFileApp.java

C:\Program Files\Java\jdk-24\bin>java CreateFileApp
File created successfully....
```

Once we create file then we can store data in file or read data from file using Java Program

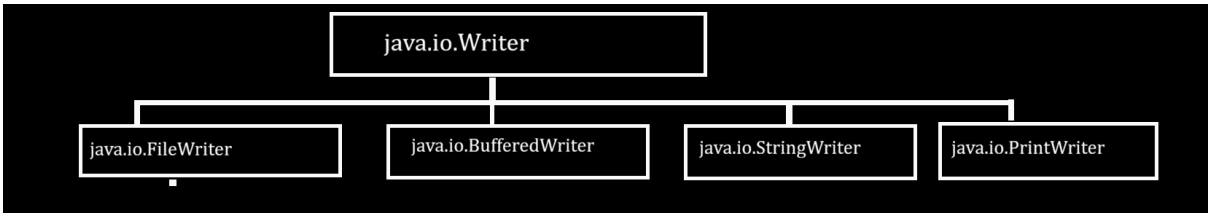
There are two ways to write data in file or read data from file

---

1. **Using a text format :** if we want to write data in text file, document file or word file etc then we can use text format
   If we want to work with text format we have Reader and Writer classes
   Means if we found any class in java ends with Reader or Writer then class design by java for work with text format
2. **Using a Byte format :** if we want to work with image, audio, video etc format then we can use byte format
   If we want to work with byte format then java provide the Stream classes to us
   Like as FileOutputStream, FileInputStream etc
   Means if we found any class in java ends with Stream then class design for work with byte format

   Now we want to discuss about Writer classes in Java

   Purpose of Writer classes is to store data in file using a text format like as notepad etc
   If we want to work with a Writer class we need to know the class hierarchy of Writer classes



   Now we want to understand the diagram

1. **Writer class:** Writer class is basically abstract class from java.io package and which is used for write data in file and it contains some abstract method which is commonly present in all writer classes and help us to write data in file

   Methods of Writer class

   ---

   public void write(int) throws java.io.IOException: this method is used for write single character at time in file and internally use ascii code of character

   public void write(char[]) throws java.io.IOException: this method is used for write data in file using character array format

   public abstract void write(char[], int, int) throws java.io.IOException: this method is used for write a specified length of data in file

   Parameter Details

   ---

   char [ ]: this parameter contain whole data
   int index: this parameter index from data want to write in file
   int length: this parameter decides how much data you want to write in a file from index.

   public void write(java.lang.String) throws java.io.IOException: this method help us to write complete string data in file

   public void write(java.lang.String, int, int) throws java.io.IOException:
   Parameter details
   String : this parameter represent data in string format
   int index: this parameter represent index from data want to write
   int length: this parameter decide how much data want to write in file

   public java.io.Writer append(java.lang.CharSequence) throws java.io.IOException: this method is used for write data or append data at the end of file
   public java.io.Writer append(java.lang.CharSequence, int, int) throws java.io.IOException: this method can append data in file using a specified index
   public java.io.Writer append(char) throws java.io.IOException: this method can append single character at the end of file

   public abstract void flush() throws java.io.IOException: this method can clear buffer of file
   public abstract void close() throws java.io.IOException: this method is used for close the file

   Now we want to discuss about FileWriter class

FileWriter class: FileWriter class is used for write data in file using a text format but in single line and this class create new file if file is not exist or append existing file
FileWriter class or Any file writing class work with two modes

Write mode: Write mode means if file is not exist then create new file and if file is exist then override previous content of file and add new content in file

Append mode: Append mode means if file not exist then create new file and if file is exist then save or persist previous content of file and add new content at the end of file

Constructor of FileWriter class

FileWriter(String path) : this constructor help us accept file path using a string format and use write mode
FileWriter(File path) : this constructor help us to accept file path using a File class reference and use write mode
FileWriter(String path,boolean mode)
String path : this parameter help us accept the file path
boolean mode: this parameter decides whether the file can use write mode or append mode. If we pass true then file can use append mode and if we pass false then file can use write mode.
FileWriter(File path, boolean mode):
File path : this parameter help us accept the file path
boolean mode: this parameter decides whether the file can use write mode or append mode. If we pass true then file can use append mode and if we pass false then file can use write mode.

Example: now we want to create text file and store data in it using a FileWriter class

```
import java.io.*;
import java.util.*;
public class SaveFileApp
{
    public static void main(String x[])throws Exception
    {
        FileWriter fw=new FileWriter("D:\\janfebmarch\\notes.txt",true);

        Scanner xyz = new Scanner(System.in);
        System.out.println("Enter data in file");
        String data=xyz.nextLine();

        fw.write(data);//we write string in file
        fw.write("\n");
        fw.close();
    }
}
```

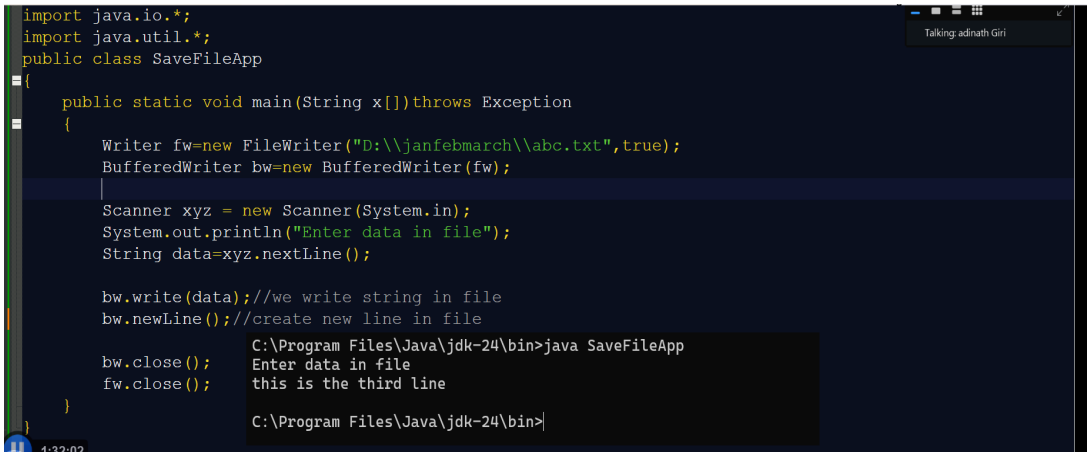
Note: if we think about FileWriter by default FileWriter can add data in single line we have to add data on new line using \n manually
But if we want to write data in file on new line we have one more class name as BufferedWriter class

Now we want to discuss about BufferedWriter class

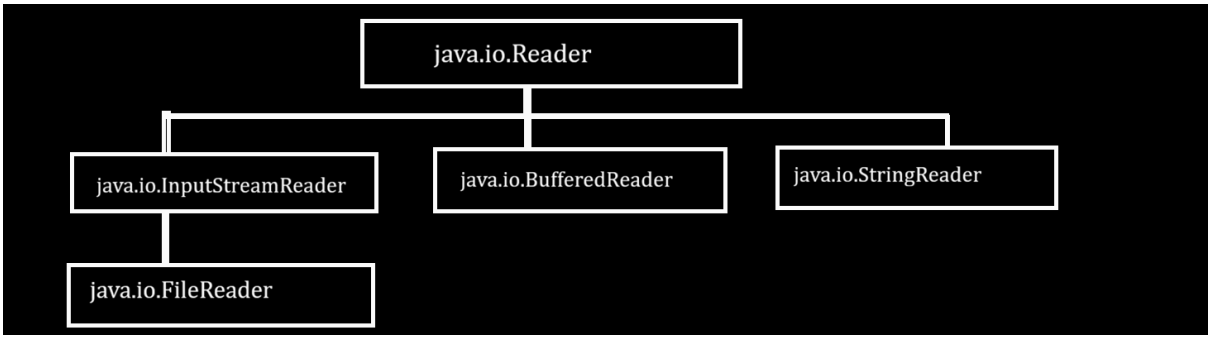
BufferedWriter: BufferedWriter class is used for add data on new line in file and for that purpose BufferedWriter class provide newLine() method to us

Constructor of BufferedWriter

BufferedWriter(Writer): this constructor accept reference of Writer class means can accept reference of any child of Writer or use Writer using upcasting technique.



Reader classes: Reader classes is used for read data from file using a text format
If we want to work with Reader classes we have following class hierarchy



If we think about above diagram we have java.io.Reader class and it is abstract class and parent of all text data reading classes and it contain some abstract method which help us read data from file

Methods of Reader class

---

public int read() throws java.io.IOException: this method is used for read single character at time from file and return -1 when file is end or when file has no data

public int read(char[]) throws java.io.IOException: this method can read all data from file and store in character array and return its length if file contain data otherwise return -1

public abstract int read(char[]data, int offset, int length) throws java.io.IOException: this method can read specified data from file

char[]data: this parameter contain data
int offset: this parameter indicate index from data want to read
int length: this parameter decide actual count of data want tor ead

public void mark(int) throws java.io.IOException: this method is used for set index from data want to read and index start from 0

public void reset() throws java.io.IOException: we can reset pointer at beginning of file
public abstract void close() throws java.io.IOException: this method help us to close the file

FileReader class

FileReader class is used for read data using character format

Constructor of FileReader class

FileReader(String path): this constructor is used for accept file path using a string format and use by default read
FileReader(File path): this constructor is used for accept file path using file class reference format

Example: Read file using a single character at time

```
import java.io.*;
import java.util.*;
public class ReadFileApp
{
    public static void main(String x[])throws Exception
    {
       FileReader fr=new FileReader("D:\\janfebmarch\\abc.txt");
       int data;
       while((data=fr.read())!=-1)
       {
           System.out.print((char)data);
           Thread.sleep(100);
       }
    }
}
```

```
C:\Program Files\Java\jdk-24\bin>javac ReadFileApp.java

C:\Program Files\Java\jdk-24\bin>java ReadFileApp
this is the first line
this is the second line
this is the third line
```

Note: if we think about above code we read data from file using character by character But if we want to read data from a file using line by line we have to use the BufferedReader class.
Constructor of BufferedReader class

BufferedReader(Reader): In BufferedReader class constructor contain reference of Reader class means we can pass reference of any child of Reader class or Reader reference itself by using upcasting
Note: BufferedReader provide readLine() method to us which is used for read data line by line and return null value when file is finish or file has no data

Example with source code

```
import java.io.*;
import java.util.*;
```

```
public class ReadFileApp
{
   public static void main(String x[])throws Exception
        {
          FileReader fr=new FileReader("D:\\janfebmarch\\abc.txt");
          BufferedReader br= new BufferedReader(fr);
          String  data=null;
          while((data=br.readLine())!=null)
          {
                  System.out.println(data);
                  Thread.sleep(100);
          }
        }
}
```

Stream classes : Stream classes are used for store data in byte format as well as read data in byte format . Stream classes normally recommend to work with image, audio,video etc

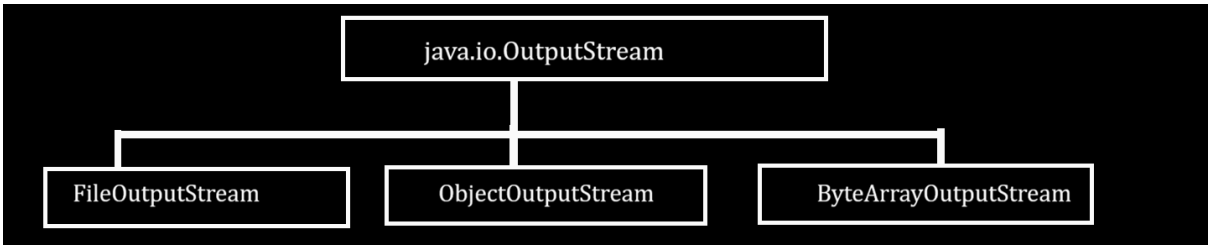
There are two types of Stream classes

---

a. **OutputStream:** OutputStream is used for store data in byte format
b. **InputStream :** InputStream is used for read data in byte format

Now we want to discuss about OutputStream class

Hierarchy of OutputStream class



If we think about OutputStream class it is abstract class and it contain some abstract methods which help us to write data in byte format

Methods of OutputStream class

---

public abstract void write(int) throws java.io.IOException: this method is used for write data in byte format and write single byte at time
public void write(byte[]) throws java.io.IOException: this method is used for write byte array data in file
public void write(byte[], int, int) throws java.io.IOException: this method is used for write specified length of data in file using a byte format
byte[]: this parameter indicate actual data we want to write in file
public void flush() throws java.io.IOException: this method can clear your buffer
public void close() throws java.io.IOException: this method is used for close the file

FileOutputStream: this class is used for store data in byte format

Constructor of FileOutputStream class

FileOutputStream(String path); accept the file path using a string format and use write mode

FileOutputStream(String path,boolean mode): this constructor is used for accept file path using a string format and use append mode if we pass true value then use append if we pass false value then use write mode

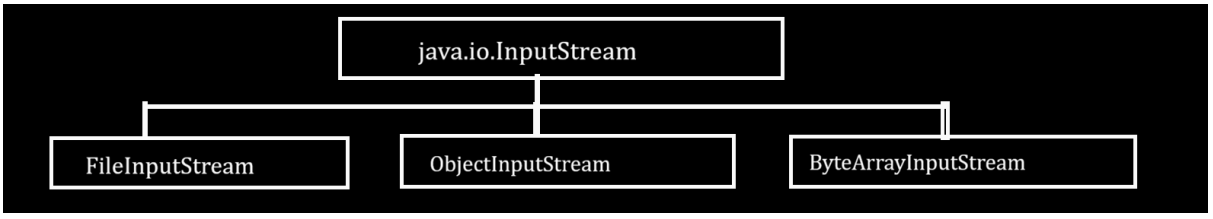
FileOutputStream(File path): accept the file path using a file class reference and use write mode by default

FileOutputStream(File path,boolean mode): this constructor is used to accept file path using a file class reference and use append mode if we pass true value in it otherwise use write mode.

Example: WAP to input string and store in file using a FileOutputStream

```
import java.io.*;
import java.util.*;
public class SaveDataApp
{   public static void main(String x[])throws Exception
        {  Scanner xyz = new Scanner(System.in);
           FileOutputStream fout=new FileOutputStream("D:\\janfebmarch\\abc.txt");
                System.out.println("Enter data in file");
                String data=xyz.nextLine();
                byte b[]=data.getBytes();
                fout.write(b);
                fout.close();
                System.out.println("Data Save");
        }
}
```

InputStream: InputStream class is used for read data from file using a byte format



Here InputStream is abstract class and it is parent of all input stream classes

Methods of InputStream class

---

public abstract int read() throws java.io.IOException: this method is used for read single byte at time from file and return -1 when file is end

public int read(byte[]) throws java.io.IOException: this method reads all file data and stores it in a byte array and returns its length otherwise return -1.

public int read(byte[], int, int) throws java.io.IOException: this method is used for read specified length of byte data

byte[]: this parameter is used for store all data in file using byte array format
int offset: this parameter is used for set index from data want to read
int length:this parameter is used to decide how much data you want to read from a file
.
FileInputStream class : this class is used for read data from file using byte format

Constructor of FileInputStream class

FileInputStream(String path): this constructor is used for accept file path using a string format
FileInputStream(File path): this constructor is used for accept file path using a file class reference

Example: WAP to read file using a FileInputStream class

```
import java.io.*;
import java.util.*;
public class ReadFileApp
{   public static void main(String x[])throws Exception
        {
                FileInputStream fin= new FileInputStream("D:\\janfebmarch\\notes.txt");
                int data;
                while((data=fin.read())!=-1)
                {
                        char ch=(char)data;
                        System.out.print(ch);
                }
        }
}
```

Example: Read image from hard disk and store on another place e.g we copy image and paste on another location

```
import java.io.*;
public class CopyPasteImageApp
{
  public static void main(String x[])throws Exception
  {
                FileInputStream fr=new FileInputStream("D:\\dec\\abc.png");
                FileOutputStream fw=new FileOutputStream("D:\\janfebmarch\\abc.png");

                int data;
                while((data=fr.read())!=-1)
                {
                   fw.write(data);
                }
```

```
            fw.close();
            fr.close();

    }
}
```

Serialization and Deserialization

Serialization is the process where we can convert object data into byte format and store in file
Means using a Serialization we can store encrypted data in file means internally JVM encrypt your file content and store in file using a serialization

Deserialization: Deserialization is a process where we can read data from file and decrypt it in original format called as deserialization

How to implement Serialization using JAVA

If we want to implement serialization in java we have to use Serializable interface
From java.io package and Serializable interface is a marker interface in java
Marker interface means interface without methods and JVM provide some special runtime environment to them called as marker interface

Steps to work with a Serialization

---

a. **Add java.io package in application**
   import java.io.*;
b. **Create POJO class and implements Serializable interface in it**

```
import java.io.*;
class Employee implements Serializable
{
   private int id;
        private String name;
        private int sal;

        public void setId(int id)
        { this.id=id;
        }
        public int getId()
        {  return id;
        }
        public void setName(String name)
        { this.name=name;
        }
        public String getName()
        {  return name;
        }
        public void setSal(int sal)
```

```
        {  this.sal=sal;
        }
        public int getSal()
        {  return sal;
        }
}
```

c. **Create Object of ObjectOutputStream class :** Here ObjectOutputStream is a class from java.io package which is used for store data in file using byte format Means this class accept object data and store in file and encrypt them or send in network
   Syntax: ObjectOutputStream ref=new ObjectOutputStream(OutputStream)
   OutputStream out=new FileOutputStream("D:\\janfebmarch\\emp.txt");
   ObjectOutputStream oout=new ObjectOutputStream(out);

d. **Create Object of POJO class and store data in it**
   Employee emp = new Employee();
   emp.setId(1);
   emp.setName("ABC");
   emp.setSal(10000);

e. **Save the POJO class object in file using write() method of ObjectOutputStream class**
   If we want to save object in file or send in network using ObjectOutputStream class we have method name as void writeObject(Object);

Syntax: **void writeObject(Object): this method is used for write any kind of data in file Or any class object in file e.g Employee**

Example with source code

```
import java.io.*;
class Employee extends Object implements Serializable
{   private int id;
        private String name;
        private int sal;

        public void setId(int id)
        { this.id=id;
        }
        public int getId()
        {  return id;
        }
        public void setName(String name)
        { this.name=name;
        }
        public String getName()
        {  return name;
```

```
        }
        public void setSal(int sal)
        {  this.sal=sal;
        }
        public int getSal()
        {  return sal;
        }
}
public class EmpSerApplication
{
    public static void main(String x[])throws Exception
         {
           OutputStream out=new FileOutputStream("D:\\janfebmarch\\emp.txt");
                ObjectOutputStream oout=new ObjectOutputStream(out);

                Employee emp = new Employee();
                emp.setId(1);
                emp.setName("ABC");
                emp.setSal(10000);

                oout.writeObject(emp);
                oout.close();
                out.close();
         }
}
```

Deserialization: if we read file data stored by using ObjectOutputStream class called as Deserialization

If we want to work with Deserialization we have class name as ObjectInputStream class ObjectInputStream class contain readObject() method which help us to read data from file and decrypt its content and get output in original format

Constructor of ObjectInputStream class

ObjectInputStream(InputStream): accept reference of any child of InputStream

```
import java.io.*;
class Employee extends Object implements Serializable
{   private int id;
        private String name;
        private int sal;

        public void setId(int id)
        { this.id=id;
        }
        public int getId()
        {  return id;
        }
        public void setName(String name)
        { this.name=name;
```

```
        }
        public String getName()
        {  return name;
        }
        public void setSal(int sal)
        {  this.sal=sal;
        }
        public int getSal()
        {  return sal;
        }
}
public class EmpSerApplication
{
    public static void main(String x[])throws Exception
         {
            InputStream in=new FileInputStream("D:\\janfebmarch\\emp.txt");
                 ObjectInputStream oin=new ObjectInputStream(in);
            Object obj=oin.readObject();
                 if(obj!=null)
                 { Employee e=(Employee)obj;
                System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
                 }
                 else{
                          System.out.println("Object not found");
                 }

         }
}
```

transient keyword: transient keyword is used for skip the data from serialization
Means if we use transient keyword with any variable then JVM not store its data in file or send in network or not encrypt its content